

PROJECT OVERVIEW & ANALYSIS

The goal of this project was to develop an embedded version of the classic Dinosaur Game on an STM32 microcontroller. Similar to the offline game in Google Chrome, it features a dinosaur that must jump over or duck under obstacles that appear on the screen. The game is rendered on two SSD1306 OLED displays, with one displaying the gameplay and the other displaying the scoreboard, and controlled by a 4×3 keypad.

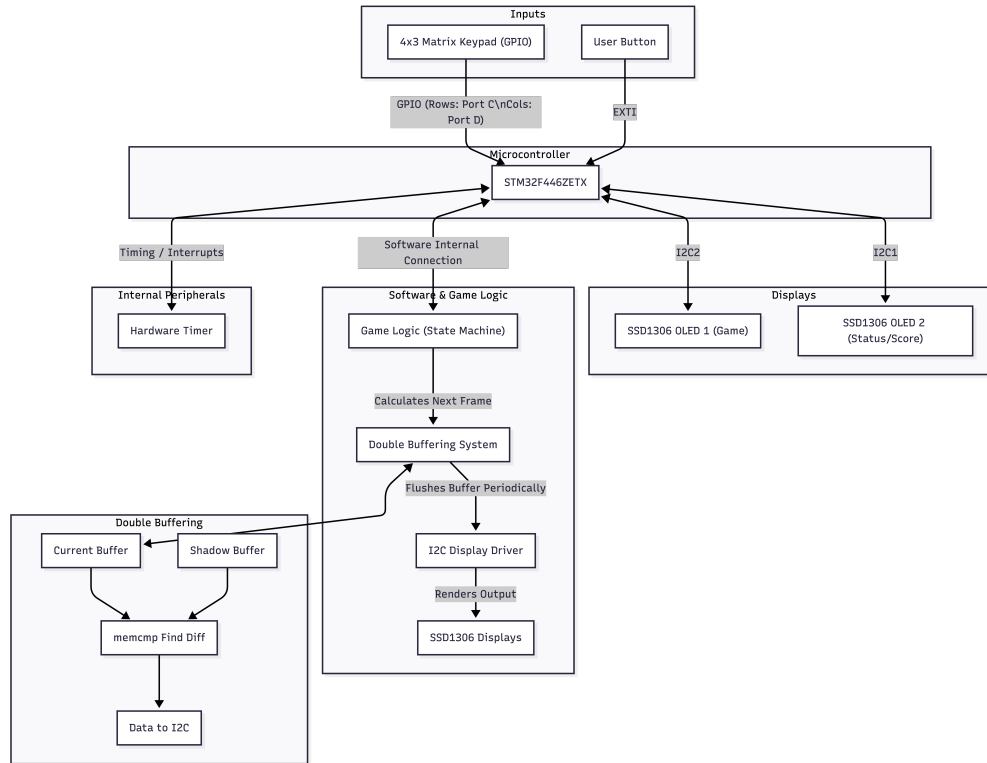


Figure 1: Block Diagram

The system is built around the STM32F446ZE Nucleo Board, which serves as the main controller of the project. It processes keypad input, executes the main game loop, and communicates with both OLED displays. The project uses two SSD1306 displays with different roles: D1 is the game display, which renders the dinosaur, obstacles, and other gameplay content, while D2 is the scoreboard display, which shows the current score, high score, and game-over status. User input is provided through a keypad with GPIO mapping, where key 2 is assigned to the jump function, and key 8 is assigned to the duck function, allowing the player to control the dinosaur's movement. The gameplay loop handles the core game logic, including object movement, obstacle updates, collision checks, score updates, and game state management, and returns the coordinates of game objects together with the current status and score. The rendering functions are based on a double-buffering structure to support smoother display updates. At the hardware level, GPIO peripherals are used for keypad signal interrupts, while I2C1 and I2C2 are used to communicate with the two OLED displays. These I2C peripherals transfer the differences between the previous frame and the new frame to the displays in order to update the output efficiently.

The game's core logic centers on dinosaur movement, obstacle spawning, and collision detection.

Keypad input controls the dinosaur: key ‘2’ for jump and key ‘8’ for duck. Gravity pulls the dinosaur down unless it jumps, gaining upward velocity to clear obstacles. Ducking reduces height to pass under low obstacles like birds. Obstacles (cacti and birds) spawn randomly and move toward the dinosaur from the right, requiring the player to jump or duck to avoid collision. Collision is detected by checking the bounding box overlap. The score continuously increases with survival; game difficulty and speed rise with the score, making obstacle avoidance faster and harder. Collision with an obstacle triggers a game-over, recording the high score and displaying the final score.

BANDWIDTH / THROUGHPUT ANALYSIS

1. I2C Interface (SSD1306 OLED Displays)

- **Limiting Parameter:** Bus speed and framing overhead.
- **Theoretical Maximum:** The I2C buses (`hi2c1` and `hi2c2`) are configured for Fast Mode at 400 kHz. An SSD1306 display (128x64 pixels) requires sending 1,024 bytes of pixel data plus a few bytes of command overhead per frame—totalling roughly 8,400 bits. Theoretically, at 400 kbps, the maximum refresh rate is approximately 47 frames per second (FPS) per display if we need to transmit every pixel for each frame. If consider using double buffering to reduce the number of pixels that needed to be transmitted per frame to only those on the page that had changed (reducing it to about $\frac{1}{8}$, for a total of 8 pages), the maximum FPS can be 300+ if no manual delay was introduced, but we cannot estimate the average frame rate because the visual changes in the game are highly variable.
- **Actual Performance:** In practice, the effective framerate is estimated closer to 15–20 FPS without double buffering, and close to 55 FPS with double buffering, which is still some gap to reach a stable 60 FPS.
- **Comparison & Explanation:** The theoretical limit is not reached because the software employs blocking `HAL_I2C_Master_Transmit` functions (implied by typical `SSD1306_UpdateScreen` implementations). The CPU halts execution to handle byte-by-byte polling transfers. Furthermore, the second screen, which displays scores and status, is exclusively text, and since the text changes significantly with every frame, this screen requires the transmission of a large number of pixels that have changed. This directly results in a significant reduction in the performance benefits of double buffering.

2. GPIO Matrix Keypad

- **Limiting Parameter:** Input update latency and debouncing loop overhead.
- **Theoretical Maximum:** The STM32F446ZE GPIO pins driven by a 168 MHz system clock can technically toggle and read at speeds in the tens of megahertz (nanosecond latency).
- **Actual Performance:** Scanning all 4 rows takes roughly 0.3 to 0.5 milliseconds per complete scan cycle.
- **Comparison & Explanation:** This massive gap between theoretical CPU speed and actual latency is an intentional, physical limitation. In `keypad.c`, a stupid but useful blocking delay (`for (volatile int d = 0; d < 3000; d++)`) is artificially injected after driving each row HIGH. This is necessary because the physical keypad matrix has parasitic capacitance and resistance; reading too fast would result in sensing incomplete voltage states.

3. CPU & Game Logic (STM32F446ZE at 168 MHz)

- **Limiting Parameter:** Clock rate and cycles per task.
- **Theoretical Maximum:** The Cortex-M4 core operating at 168 MHz executes millions of instructions between frame rendering intervals, easily capable of highly complex physics engines.
- **Actual Performance:** The implementation of Q16.16 fixed-point gravity physics, state updates, and Axis-Aligned Bounding Box (AABB) collision checks computes in under a millisecond. Consequently, CPU computation time accounts for less than 1% of the game loop duration.
- **Comparison & Explanation:** The CPU's potential is heavily underutilized. Since memory operations are fast, the system is prevented from running optimally not by logic complexity, but entirely by I/O constraints (waiting for the displays to update).

Main Bottleneck: The primary bottleneck in this system is the **blocking software handling of the 400 kHz I2C bus**. Since the CPU is synchronously waiting for data to transfer 1,024 bytes per display per frame, it wastes thousands of clock cycles where it could be handling other logic or entering a low-power state.

Greatest Improvement: The single change that would most improve performance is substituting the blocking I2C calls with **I2C Direct Memory Access (DMA)** (e.g., `HAL_I2C_Master_Transmit_DMA`). Doing so would allow the CPU to dispatch a screen update asynchronously and immediately proceed to read input and compute the next frame, unlocking the true theoretical framerate of the 400 kHz bus and drastically reducing wasted CPU cycles. Alternatively, switching to SPI-based SSD1306 displays would lift the bus speed limit entirely (SPI can operate well over 10 MHz versus I2C's 400 kHz).

CODE CONTRIBUTION TABLE

Module	Source	Notes
Gameplay Logic	From scratch	We first implemented the code on PC and tested the code, then integrated the gameplay functions into the project.
Sprites Patterns	AI Generated	AI-generated C bitmap arrays, which are used to render game player characters (dinosaurs) and obstacle graphics
I ² C	Modified from lab2 code	Modified the I2C functions, upgraded to support simultaneous data transmission via I2C1 and I2C2 to two different screens.
SSD1306 peripheral interface	Modified from lab 4 code	Only used the peripheral interface from lab 4 code, created new functions like <code>drawBitmap</code> from scratch, which allows drawing patterns for each frame
Dual display	From scratch	Display 1 for gameplay, display 2 for scores, and content are synced.
Double Buffering	From scratch	Implemented a memory compare function that can find the different pixels from two buffers and only update the different pixels; this can save a lot of bandwidth on I2C

PROJECT REFLECTION

The original goal of our project was to build a Dinosaur game on the STM32F446ZE that used double buffering, fixed-point computation, and DMA-assisted display transfer to improve rendering efficiency. We wanted the system to feel responsive on real hardware, with smooth animation on the OLED displays and clean separation between gameplay and score output. From the beginning, the main technical focus was not just making the game run, but exploring whether embedded optimization techniques normally used in more data-heavy systems would provide a meaningful benefit in this project.

We achieved most of that goal, but not all of it. The final system successfully implemented the game itself, including keypad input, obstacle generation, collision detection, score tracking, two OLED displays, and smooth visual updates through double buffering. The implementation also remained lightweight enough for the STM32 platform. However, we did not ultimately rely on DMA in the final design. After testing our rendering pipeline, we found that the drawing workload of this project was relatively simple: the graphics are low resolution, the objects are small, and the amount of screen data updated in each frame is limited. In practice, DMA did not provide a clear performance benefit compared with the extra design complexity it introduced. In addition, DMA-based display transfer would require careful scheduling and synchronization, since refreshing the displays too quickly could create unnecessary bus activity and make frame timing harder to control. For this project, double buffering alone was enough to keep the display smooth and visually stable, so we chose it as the main optimization.

If we repeated the project, one specific technical decision we would change is that we would evaluate the usefulness of DMA much earlier and more quantitatively before committing design effort to it. We spent time considering DMA as a major optimization, but later realized that the real constraint was not memory movement inside the microcontroller as much as the display communication path and the relatively modest rendering demand of the game. In a second iteration, it would make more sense to profile the full frame-update path first and then decide whether DMA, a different communication interface, or a simpler software architecture would provide the best return.

Two relevant safety issues in our project are electrical safety and fail-safe system behaviour. Because our design uses multiple external components, including two OLED displays and a keypad connected to the STM32 board, incorrect wiring, wrong pin configuration, or incompatible voltage levels could damage the microcontroller or peripherals. To reduce this risk, we would carefully verify the pin mappings, confirm voltage compatibility, and test each hardware module separately before full system integration. Another important issue is fail-safe behaviour. If the display communication fails or the game state enters an invalid condition, the system could freeze, show incorrect output, or stop responding to input. We would address this by adding bounds checking, basic error handling for communication failures, and reset or recovery logic so the system can return to a safe and usable state instead of remaining stuck.

WORKLOAD SPLIT

Haozhe focus on hardware connection, gameplay logic, fixed point gravity computation, SSD1306 peripheral and `drawBitMap` functions and dual display implementations.

Xindi works on keypad integration, double buffering implementation and system testing, and tried out DMA implementation.